Introducción a Angular

mediante ejemplos

Preparación de entorno

Prerrequisito: node (y npm) o nvm

Instalar angular: npm install -g @angular/cli

Nuevo proyecto: ng new mi-proyecto

Iniciar: ng serve [-p 4200] [-o]

-p para indicar el puerto de escucha

-o para abrir la página del proyecto en el navegador

[...] es opcional

Componentes



Directivas estructurales

```
<div *ngIf="personas">
    <h1>Listado de personas</h1>
    <div *ngFor="let p of personas">
        <p>{{ p.nombre }}</p>
    </div>
</div>
```

Listado de personas

Carlos

Pedro

Esteban

// personas será una variable.

// El div con *ngIf se mostrará si personas se

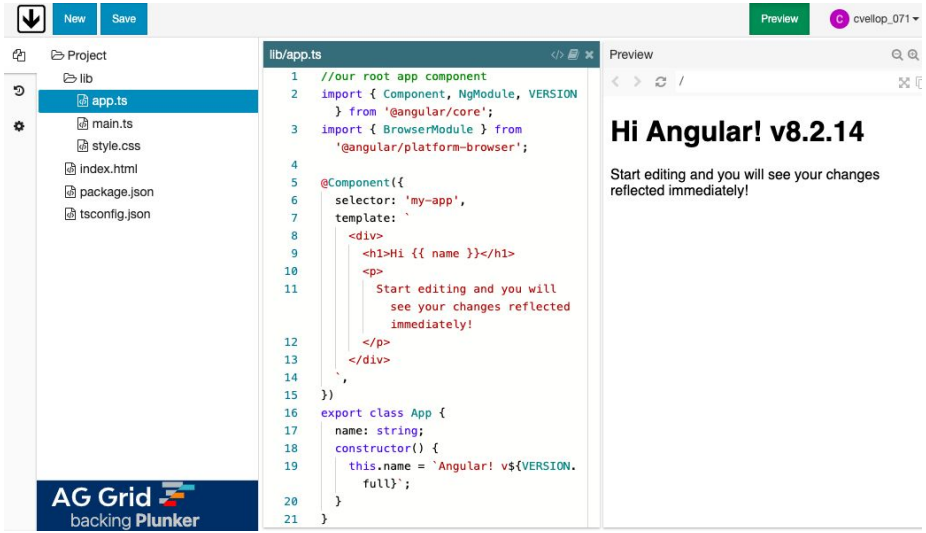
// evalúa como true.

// En div con *ngFor, se añadirá un <p> por cada

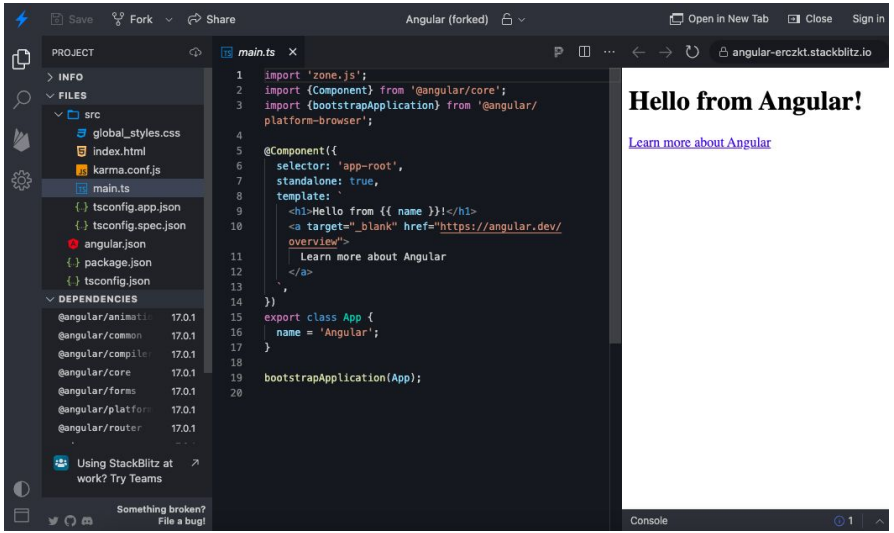
// persona (elemento) que tenga personas (array).

Entornos online de pruebas

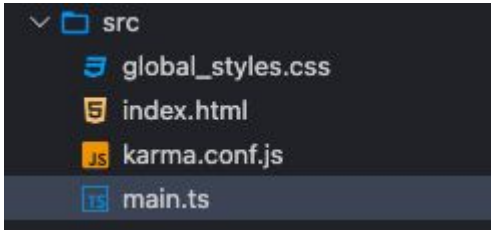
Plunker



Stackblitz



Interconexión de archivos: componentes.



Decorador

index.html

```html
<!DOCTYPE html>
<html lang="en">
  <head>
    <title>My app</title>
    <meta charset="UTF-8" />
  </head>
  <body>
    <app-root>Loading...</app-root>
  </body>
</html>
```

main.ts

```ts
import 'zone.js';
import { Component } from '@angular/core';
import { bootstrapApplication } from '@angular/platform-browser';

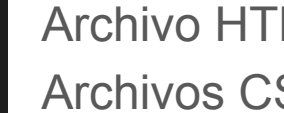
@Component({
  selector: 'app-root',
  standalone: true,
  template: `
    <h1>Hello from {{ name }}!</h1>
  `,
})
export class App {
  name = 'Angular';
}

bootstrapApplication(App);
```

HTML

Componente: usar archivos separados

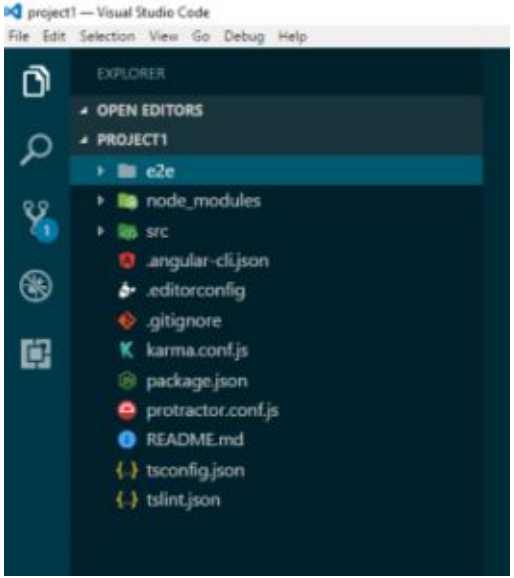
```
1  import { Component } from '@angular/core';
2
3  @Component({
4    selector: 'app-root',
5    templateUrl: './app.component.html',
6    styleUrls: ['./app.component.css']
7  })
8  export class AppComponent {
9    title = 'app';
10 }
11
```



Archivo HTML (1)

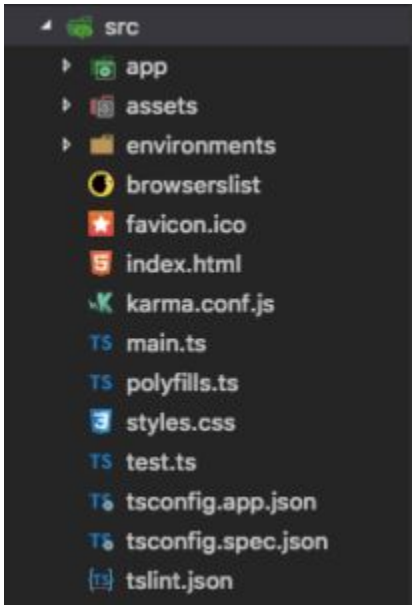
Archivos CSS (array)

Estructura del proyecto



- **e2e**: esta carpeta, denominada “end to end”, engloba unas series de ficheros cuya función es la realización de test automáticos, como si un usuario interactuase con la app. Se ejecuta con el comando ng e2e.
- **node_modules**: es la carpeta que contiene todas las dependencias de nuestro proyecto.
- **Src**: es el directorio donde trabajaremos nuestros módulos. Además, es el más importante, ya que contiene todo el código.
- **.editorconfig**: es la configuración de nuestro editor de código.
- **.gitignore**: son las carpetas o archivos que debe ignorar el git cuando lo añadamos al repositorio.
- **angularjson**: contiene la configuración de Angular. Además, incluye rutas, versiones, etc.
- **package.json**: es la configuración de nuestra aplicación. Contiene el nombre de la app, las dependencias necesarias para su correcta ejecución y muchas otras cosas.
- **README.md**: aquí podemos añadir información sobre la aplicación. Este archivo es leído por GIT y los muestra en el repositorio.
- **tsconfig.json**: contiene la configuración TypeScript.
- **tslint.json**: se utiliza para que el código sea sostenible y se mantenga.

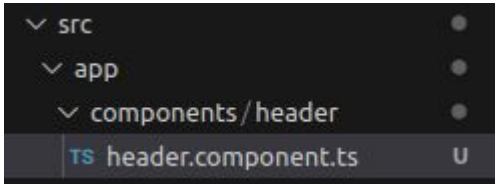
Estructura del proyecto



- **Carpeta app:** Aquí se encuentra la carpeta app donde se ubica toda la implementación de los componentes principales, junto a su template html y archivos de estilos css.
- **Carpeta assets:** Contendrá todos los asset y archivos adicionales para hacer que el proyecto funcione.
- **Carpeta enviroments:** Donde se encuentra las configuraciones y variables de entorno para poner el proyecto tanto en desarrollo como en producción.
- **Archivo favicon.ico:** Es el archivo del icono del proyecto.
- **Archivo index.html:** Es el archivo de la página principal del proyecto.
- **Archivo main.ts:** Es el archivo Type Script inicial del proyecto donde podrás configurar todas las configuraciones globales del proyecto.

Componentes: creación manual

Crear dentro de la carpeta app, una carpeta components, y dentro de esta última, una carpeta para cada componente.



Crear una carpeta header

Crear un archivo header.componet.ts

Crear una clase vacía HeaderComponent.

Añadir decorador component:

- Importar Component
- Configurar selector (app-header) y template (`<h1>Componente Header</h1>`)

Componentes: creación manual (II)

src > app > components > header > header.component.ts > HeaderComponent

```
1  import { Component } from "@angular/core";
2
3  @Component({
4      selector: "app-header",
5      template: `<h1>Componente Header</h1>`,
6  })
7
8  export class HeaderComponent {
9  }
```

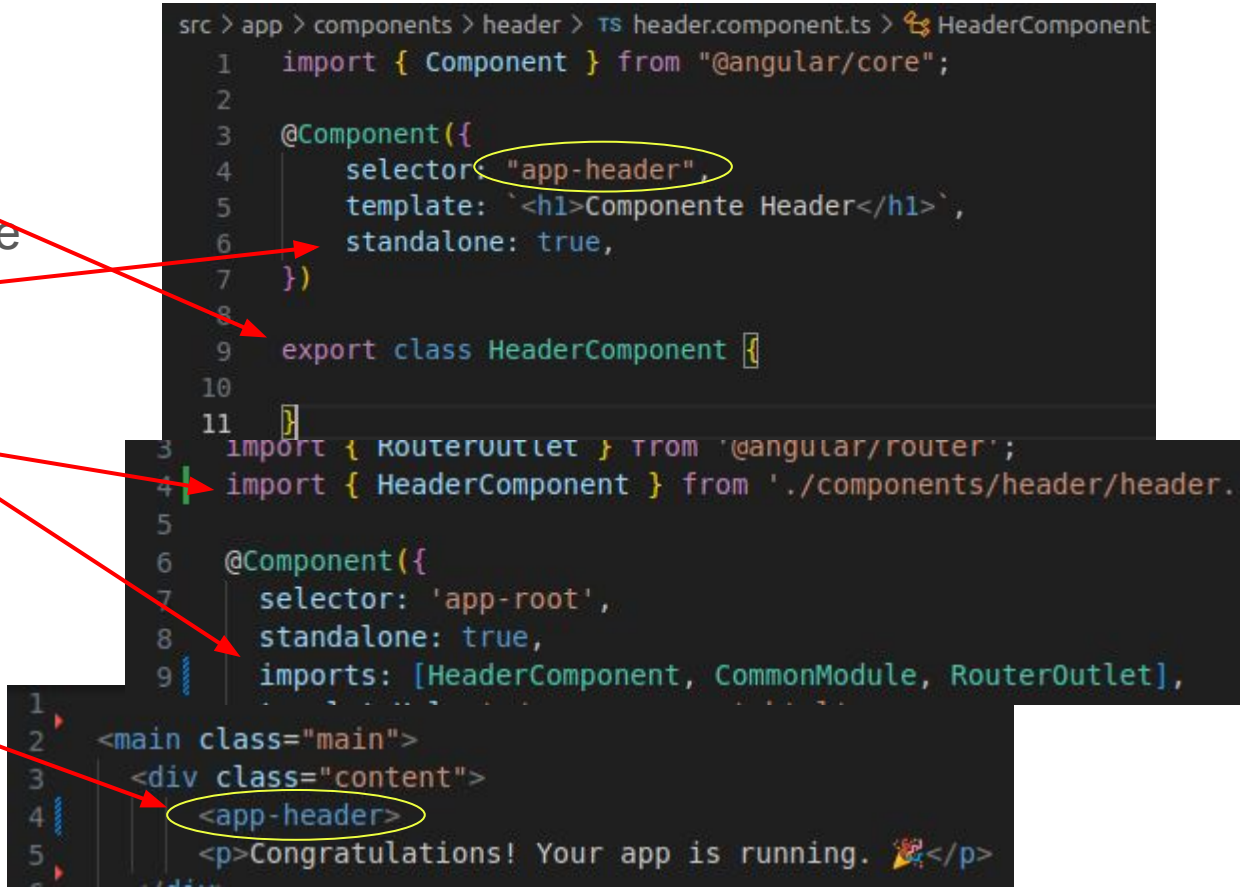
Para usar un componente:

- Exportar la clase
- Importar la clase en app.component.ts
- Incluir la etiqueta app-header en app.component.html

Componentes: creación manual (II)

Para usar un componente:

- Exportar la clase
- Declarar el componente como standalone
- Importar la clase en app.component.ts
- Incluir la etiqueta app-header en app.component.html



Componentes: creación automática

Para crear un componente **prueba**, con un archivo html y otro css:

- ng g c components/prueba

Es equivalente a:

- ng generate component components/prueba

Se crearía la carpeta **prueba** dentro de components, con los archivos:

- prueba.component.html
- prueba.component.css
- prueba.component.ts (con la etiqueta app-prueba en selector).

También se importa el componente **prueba** al archivo app.component.ts